

Instructions for using Meter7.9.py

7.9 is the latest version of Power Meter, which has several improvements over previous versions of the script.

First, it displays the energy exported as well as that used from the grid. The Pylon symbol above the export figure indicates the direction of power to and from the grid.

It has a Boost function which will provide a timed discharge from the batteries, either to feed the house or to export to the grid. It also has an improved algorithm to measure the energy imported and exported from the inverter, which overcomes a previous problem resulting from the inverter's software. The script now monitors the inverter every 15 seconds rather than every 30 seconds as in previous versions, and there is an EV mode which attempts to prevent your car emptying the contents of your house battery when it is being charged.

The Meter7.9 script, when used on a Raspberry Pi, can be run either from the Graphical User Interface (GUI) as a script under Thony or as an embedded system that runs automatically every time the Raspberry Pi is rebooted. That requires some editing of the operating system on the Raspberry Pi, so read **Using the script as an embedded system**.

Meter 7.9.py has its settings in a separate file **Settings.py**.

This file can be edited with any text editor, but please take care and make sure that text strings are enclosed in quotation marks and that your text editor does not try and correct any spellings.

For details on editing this file, read **Instructions for Settings.py** later in this document.

Meter7.9.py displays PV power, battery power, excess power to the grid and the current battery state of charge. It also displays the energy used from the grid during peak (daytime), off-peak (nighttime) periods and energy exported during the day

With the Solcast module activated, it displays an estimated overnight preload of the battery to make the best use of both PV power and battery power.

Meter7.9.py also has the ability to set the battery preload automatically, and on days with little solar yield can delay the use of batteries until the early evening. For this purpose, Meter 7.9 also learns your average usage of electricity on each day of the week.

Pressing or clicking on the cogwheel symbol  at the top left of the screen opens a control panel which enables you to change the mode of the inverter or change the set charge and discharge times. For further details, read **Using the Control Panel**.

Pressing on the timer symbol  at the bottom left of the screen opens the Boost dialogue, read **Using the Boost facility** to understand the Boost function.

The script can create a log file called **powerlog.csv**, which can be entered into a spreadsheet program for further processing.

The values logged are Solar yield, Battery use, Grid power used during daytime, Grid power used during nighttime off-peak, and Exported energy to the grid over the previous 24 hours. Note that for this purpose, the 24 hours start at the beginning of the off-peak period.

These values are not logged by the SolaX cloud, but are calculated by the script; therefore, for accurate values, the script must run continuously.

Each 24 hours starts at the beginning of the off-peak period or midnight if no off-peak period is set.

Note that when you first run the script, the first day's log may not be recorded or may be incomplete.

The script also creates four temporary files on your drive. **power.json** stores the details from the last access to the inverter and allows the script to resume after a restart. **daymeans.json** stores average daily power usage to be used by the Solcast module, and **Set_bak.json** is a backup file of your current settings. **Times.json** is a backup of your battery charge and discharge times.

If updating from a previous version of the script earlier than Meter 7.7, you must delete the old **power.json** file before running **Meter7.9.py** as the format has changed.

As well as **Settings.py**, Meter7.9 requires several external modules, all of which must be in the same directory as the main script.

It also requires a modified version of graphics.py by John Zelle, called **graphics3.py** which is included in the repository. All these are contained in the **scripts** directory in the Github repository.

To install, I recommend that you click on the green **Code** block in Github and download the Zip file, then unzip it on the Raspberry Pi. Extract the **scripts** folder to your home directory then rename it as you see fit.

Meter 7.9 can also be used on systems other than the Raspberry Pi, but the Auto mode uses Linux system commands which may create errors on other operating systems.

Using the script as an embedded system

The Meter7.9 script when used on a Raspberry Pi can be used either from the Graphical User Interface (GUI) as a script under Thony, or it can be used as an embedded system which will run automatically every time that the Raspberry Pi is rebooted.

In order to do this, some changes are required to the operating system.

First, change the 'run' line in **Settings.py** to "AUTO"

```
run = "MANUAL"                                #Change to "AUTO" if running script
                                              automatically (see manual)
```

This modifies the script slightly, including adding a twenty-second delay at startup to allow the GUI on the Raspberry Pi to fully boot up and connect to WiFi.

Next, open the command line window on the Raspberry Pi and type the following:

```
sudo nano /etc/xdg/lxsession/LXDE-pi/autostart
```

This will open the nano text editor with the autostart file.
Move your cursor to the bottom of the file and type:

```
@/usr/bin/python /home/yourname/folder/Meter7.9.py
```

Where **yourname** is the name of your home folder, **Meter7.9.py** is the current version of the script and **folder** is the folder where the script is placed.

Close the file with **Ctrl-X** and reboot your Raspberry Pi.

The script should now start about 20 seconds after the Pi has rebooted. You will note that the window now has an 'Escape' symbol  at the bottom right, pressing this will enable you to minimise the script and get back to the standard Pi GUI.

The script contains a built-in watchdog timer. If any process takes longer than one minute, the Raspberry Pi will reboot. This should not affect the operation of the script, but if, for some reason, the script does not restart after the reboot, delete the temporary files; all those ending in **.json** then reboot the Pi again. Note that this will lose your previous data, and the script will start from scratch.

Any errors which have occurred during the running of the script will be added to the file **errlog.txt**, which can be viewed in any text editor.

For further resilience, I recommend rebooting the Raspberry Pi at regular intervals. This can be done automatically. I reset the Pi at 8.50 every morning. To achieve this, do the following:

In the Pi's command line, type the following:

```
crontab -e
```

If you have not done this before, you will be given the option of which text editor to use. The default is nano.

At the bottom of the page, type the following:

```
50 8 * * * sudo reboot
```

Close the text editor and reboot.

The time can be changed to your convenience. Do not however, be tempted to reboot overnight, as the script may assume that the backup file is out of date and ignore it when starting.

Reboot either before the start or after the end of your off-peak electricity period.

I also recommend not rebooting on the hour as the script carries out some functions on the hour, which may be missed due to the reboot.

Note that when running as an embedded system, the backup and log files are created in your root directory rather than in the same directory as the script.

Instructions for Settings.py

Before you can use Meter7.9.py, you will first have to edit the settings for your solar installation.

This file can be edited with any text editor, but please take care and make sure that text strings are enclosed in quotation marks and that your text editor does not try to correct any spellings.

You will first need to set the scale for the meter display. The scale value needs to be set, in kW, to a value more than the maximum power you need to display.

For example, the default value is 5, ideal for a 4kWp solar array and a 3.68kW inverter. If your solar array is 7kW peak and your inverter is 5kW, then set **scale** to 8.

```
scale = 5 #scale of meter in kW
```

Next, you will need to know both the password and the token ID from the Solax Cloud.

Assuming that your inverter has been linked to the Cloud by your installer, first log in to the Solax Cloud website at <https://www.solaxcloud.com>.

No.	Device Name	Device SN	Registration No.	Device Type	Device Model
1	WIFI3.0		SXKEZRERR7	Dongle	WIFI3.0
2	X1-Hybrid-G4	H4372THB027028	SXKEZRERR7	Inverter	X1-Hybrid-G4

Go to the Inverter page and copy the Registration number, which starts with the letter S. This is also your password, so paste it into the password line in the script.

```
InverterPassword = 'SXXXXXXXXX' #Password to access inverter - get  
                             from Solax Cloud website
```

SolaX Power Developer Portal

Secure and Reliable Data Service

Quick Start →

Professional & Reliable | Efficient Response | Open & Compatible

Residential API (OLD)

tokenID: 20220711034990442471535 Copy

API Address: <https://global.solaxcloud.com> Copy

Next, go to the API page (under Service) where you can create and copy your Token ID. Paste this into the tokenID line in the script.

Remember to keep both the token and the password within the quotation marks.

```
tokenID = '12345678912345678912' #API Token from Solax cloud website
```

Note that the tokenID is not essential for Meter7.7.py onwards, as they do not use the SolaX cloud API.

If you have peak and off-peak electricity tariffs, put the time of the start of each into the next four lines. The default settings relate to 'Economy 7' in the UK.

If your energy supplier has a rate change which occurs other than on the hour, you can set the number of minutes after the hour.

```
startCheapRate = 0                #Start of cheap rate electricity in
                                   hours (LOCAL or UTC, See below)

crMins = 0                        #Set to minutes if cheap rate starts
                                   at a time other than on the hour

endCheapRate = 7                  #Start of full rate electricity in
                                   hours (LOCAL or UTC, See below)

prMins = 0                        #Set to minutes if peak rate starts
                                   at a time other than on the hour
```

If you do not receive off-peak power overnight, set all these values to zero.

Note that some older electricity meters are not automatically reset during the summer to cope with daylight saving time. If this is the case, you should enter the times in UTC (GMT) **not** local time and set the timeZone setting to "UTC" rather than "LOCAL".

```
timeZone = "LOCAL"                #Set to "UTC" if rate changes do not
                                   change with daylight saving
```

The peak and off-peak usage is not available from the inverter; instead, it is counted by the script itself. In order for these to be accurate, the script has to run continuously. If the script is stopped for any length of time, these values may no longer be correct.

These figures will remain blank until some usage has been measured.

If you want to use the Solcast module, change the word 'OFF' to 'ON' and read the instructions on setting up Solcast. Even if you are not using it, the Solcast module needs to be present in your directory.

Power Meter 6.7 and 7.7 onwards now also have the ability to create a log of energy used. Enter "YES" in the line 'log ='

```
log = "YES"                       #set to "YES" to log data to a text file
```

The next settings apply only to Meter7.9.py

Meter7.9 can be used in two ways. You can either log into the inverter's WiFi access point (AP) directly, or you can contact the inverter via your house WiFi.

The SSID of the inverter starts with WiFi_S followed by nine digits.

In this case, you should set the IP number in the script to '5.8.8.8'. If you have the newer Wifi module on the inverter with 10-second updates, the IP number needs to be 192.168.10.10.

Note I have recently been told that later versions of the software on SolaX inverters no longer allow access to the API via your house WiFi. In this case, you **must** operate Meter 7.9 connected to the inverter's AP. Read **Instructions for using Meter7.9.py with the new SolaX Pocket Dongle v4.0** for an explanation on how to do this.

To connect via your house WiFi, you need to know whether your inverter has been set with a static IP number or whether it is allocated by your router.

If you know that the IP address is static, then enter that address into the Inverter Address line in the script.

If not, then enter **dhcp** between the quotation marks. When you run the script, it will search your WiFi to find the inverter. If your router should change the inverter's IP address, the script will search again.

You also need to enter the DHCP range of your router on the two following lines. The default values will be correct for most domestic routers as supplied by your ISP.

```
inverterAddress = 'dhcp'          #address of inverter - use '5.8.8.8'
                                   if logged into inverter's access
                                   point

#or 192.168.10.10 if you have the later WiFi module.

#if logging into your house WiFi and the address is not static, enter
                                   'dhcp'

#Set the DHCP range of your router on the lines below

From = '192.168.1.1'              #Start of address range for DHCP
                                   allocation

To = '192.168.1.254'              #End of address range for DHCP
                                   allocation
```

Note that if the DHCP range is large, the search process may take several minutes.

In addition, Meter 7.9 has a facility which delays the start of the battery discharge phase on days of very poor solar yield so that there is still power available from the batteries in the early evening, when the grid is under the greatest load.

On certain tariffs, this will also save you money (e.g. Octopus Flux).

This function delays the discharge start time for up to eight hours, depending on the estimated daytime usage.

To enable this function, set dischargeDelay to "AUTO".

```
dischargeDelay = "AUTO"          #set to "AUTO" if you wish to use
                                   the discharge delay facility
```

Set noDelay to the earliest time that you want the batteries to start discharging. The default is 7 am. This is the setting you would normally use in the SolaX app.

```
noDelay = 7                      #set earliest time for battery
                                   discharge in hours. e.g. 8 means 8
                                   am, 13 means 1 pm.
```

The run setting should be set to 'AUTO' if you are using the script as an embedded system; otherwise, leave it as 'MANUAL'.

The next settings are for the use of the Octopus functions for those who use Octopus as their electricity supplier. For details of these, read the instructions for the Octopus module.

The final settings are for the use of the Solcast module. Read the relevant instructions to set these.

Finally, save the edited file to the same folder as the main script, keeping the name **Settings.py**

Instructions for use of the Solcast module

The Solcast module **Solcast4.py** downloads the next day's estimated solar yield from toolkit.solcast.com.au and uses it to calculate how much energy to add to the home batteries on an overnight off-peak rate to make the best use of the batteries and solar panels.

The first requirement is to go to the Solcast website and sign up. You then need to enter your precise location and the details of your solar array.

Once you have done that, you need to select your site, which will take you to the page showing a graph of estimated solar yield.

In *Site Summary*, you will see a Resource ID. Click on this, and it will be copied to your clipboard. Paste into Settings.py in the Resource ID line, remembering to put it within the quotation marks.



Site Summary	
Resource Id	dbc0-3309-b81d-3582
Latitude	52.238716
Longitude	0.914375
AC Capacity (Inverters)	3.8 kW
DC Capacity (modules)	4 kW
MORE 	

```
resourceID = 'XXXX-XXXX-XXXX-XXXX' # obtain from Solcast website
```

In the download section near the bottom of the page, select **Find your API key here**. Click **Copy Key**, then paste it into the solcastKey line in Settings.py

Download data from the API

Helpful information

[Check out the API documentation](#)

[Learn about our forecast parameters](#)

All our data uses UTC timestamps - (ISO 8601 standard)

[Find your API key here](#)

[Learn how to use your API key](#)

['Live' vs 'Forecast' data](#)

[You can find descriptions of the included parameters on the Rooftop Solar product page](#)

Live

Forecasts

Time granularity (mins)

30

Format

JSON

CSV

XML

API

https://api.solcast.com.au/rooftop_sites/dbc0-3099-b81c-3582/forecasts?format=json

Copy Url

Download JSON

```
solcastKey = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ123456' # obtain from Solcast website
```

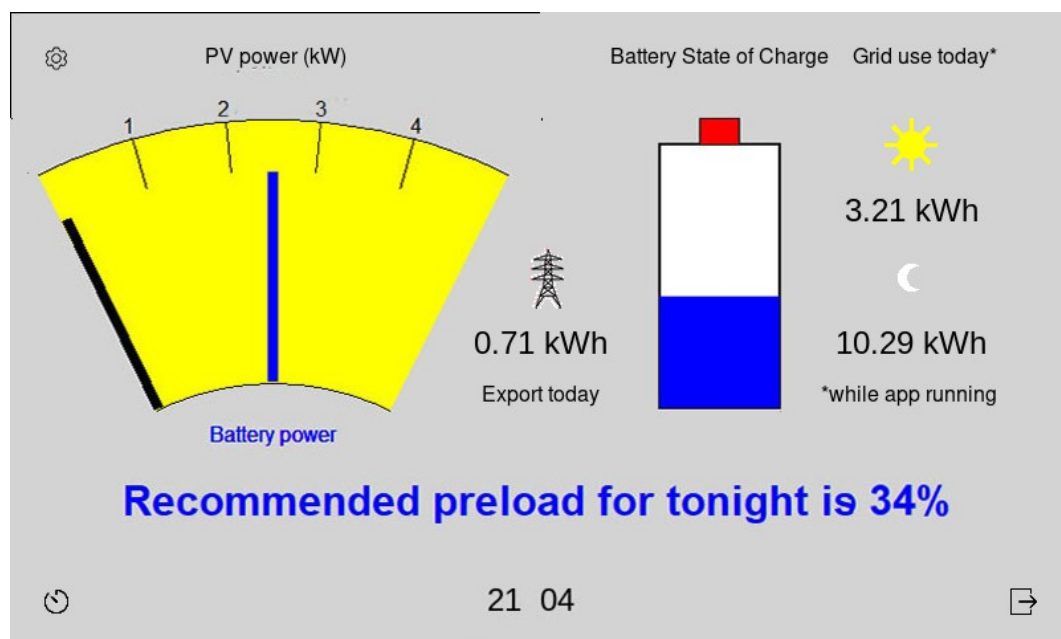
Finally, enter the capacity of your home batteries in kWh.

```
capacity = 11.6 # Capacity of battery in kWh
```

In the settings file, set the solcast line to ON

```
solcast = "ON" #Set to 'ON' if using Solcast to forecast overnight preload (see notes)
```

Each evening at 9 pm local time, the script will obtain the latest forecast from Solcast and calculate the recommended preload for the batteries for that night.



The script will continue to display the result until 1 a.m., allowing plenty of time to change the setting on the inverter.

If **preloadSet** is set to 'AUTO', then scripts Meter7.7.py and later will automatically set the preload for the batteries.

```
preloadSet = "AUTO"           #Set to "AUTO" to allow app to
                                automatically set the overnight charging limit.

offset = 30                    #percentage offset added to
                                recommended preload
```

A percentage offset can be added to the preload setting in order to account for local differences to the prediction, e.g. partial shading by a tree.

dischargeDelay if set to AUTO will allow the script to set a delay of up to eight hours to the use of the batteries on days of low sun, in order that power is retained for use in the peak hours in the evening, rather than using all of the battery power first thing in the morning.

```
dischargeDelay = "MANUAL"      #set to "AUTO" if you wish to use
                                the discharge delay facility

noDelay = 7                    #set earliest time for battery
                                discharge in hours. e.g. 8 means 8 am, 13 means 1 pm.
```

noDelay sets the earliest time that the batteries will discharge on days of good sunlight.

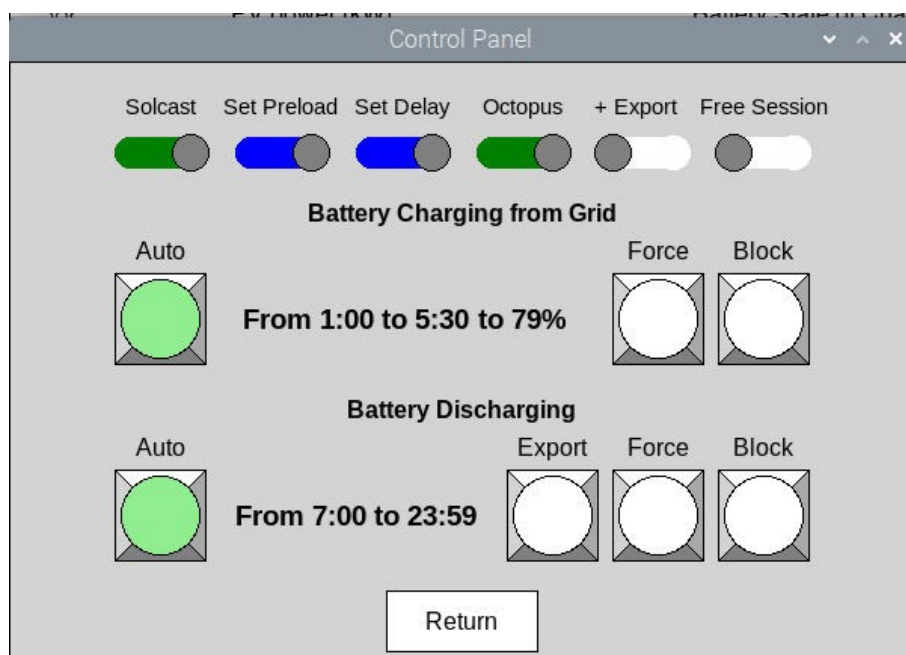
The calculation also uses an estimate of your likely use of power during the next day. The estimate is based on previous use for that day of the week.

It will probably take a few weeks to generate a reasonably accurate estimate. The calculation is based on filling the battery by 3pm local time.

I take no responsibility for the accuracy or otherwise of Solcast's predictions.

Using the control panel

At the top left of the meter display, there is a cog wheel symbol. Pressing and holding the screen at that point will bring up the control panel.



The control panel consists of one row of switches at the top and two rows of buttons. The row of switches at the top control features of the application, and the two lower rows control the inverter directly.

The first switch at the top turns on and off the Solcast function, which uses a prediction of the next day's solar yield to decide how much to preload the battery overnight and how soon to start using the battery to feed the house (read Solcast instructions).

When the switch is on (to the right and Green) the script will download the Solcast prediction for tomorrow's solar yield each evening at 9 pm.

The next two switches control whether the Solcast module should directly control the overnight precharge and the delay before the battery is used the next day.

If **Set Preload** is turned off (left – White), then the app will simply display the recommended preload but not change the setting.

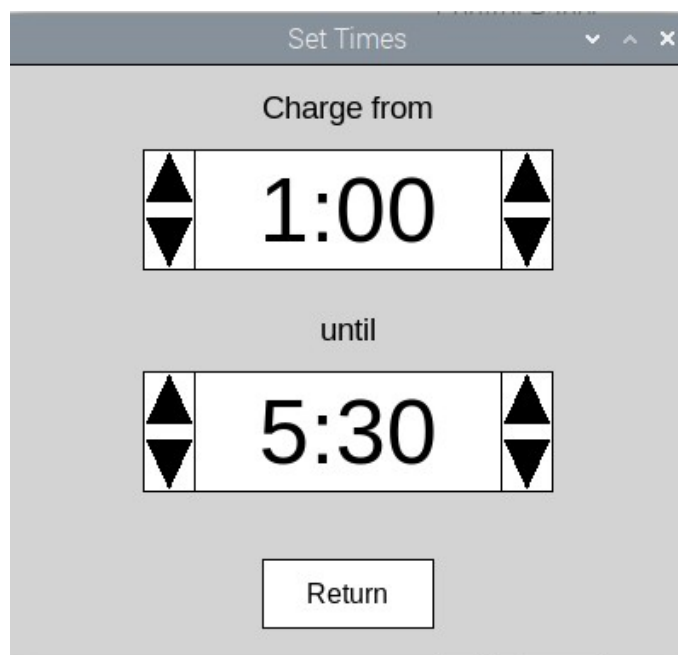
If **Discharge Delay** is on (Blue), the app will also set the time that the battery will begin to be used the next day.

The three remaining switches are for people who are receiving their grid power from Octopus Electricity. Read the Octopus instructions for further information.

To toggle a switch, simply touch it with your mouse pointer or finger if using a touch screen.

The top row of buttons controls the charging of the battery from the power grid.

Normally, the **Auto** button will be pressed (Green), allowing the inverter to switch automatically to charge at the times shown. Touching these times will open a panel with 'nudge' buttons to allow you to change the charge times.



The screenshot shows a 'Set Times' dialog box. It has a title bar with the text 'Set Times' and standard window controls (minimize, maximize, close). The main area is light gray. At the top, it says 'Charge from' above a digital time display showing '1:00'. The display is flanked by two vertical arrows (up and down) on each side. Below this, it says 'until' above another digital time display showing '5:30', also flanked by two vertical arrows. At the bottom center is a white button with the text 'Return'.

Tapping on the up and down arrows will set the times. The up-arrow on the right moves the time forward by ten seconds. The down-arrow moves backwards by one second, so any time can be set fairly easily. The left-hand arrows adjust by a complete hour. Press **Return** to return to the Control Panel.

Touching the charge percentage will open a similar 'nudge' panel to allow you to change the percentage overnight precharge.

A little tip: do not set the discharge finish time to midnight (00:00) or later, as that can confuse the inverter.

When making any changes, the **Return** button will turn Red and say **Wait** whilst the instructions are sent to the inverter. This may take up to 30 seconds.

Once it returns to White press **Return** to close the Control Panel.

Pressing the **Force** button (illuminated in Blue) will force the batteries to charge from the Grid until they either reach 100% charge or you press the **Auto** button again.

Pressing the **Block** button (illuminated in Red) will prevent the batteries from charging from the grid until you press either the **Auto** or **Force** buttons.

The batteries will always charge from solar power, whichever button is pressed.

The bottom row controls the battery discharge.

You can adjust the discharge times in a similar way to the charge times.

Pressing **Block** will prevent the batteries from discharging

Pressing **Force** will force the batteries to discharge to the house.

Pressing **Export** will force the batteries to discharge, but any excess power not required by the house will be sent to the grid.

Do not forget to press **Return** to close the Control Panel after you have used it as that will allow the script to continue.

Settings made in the Control Panel are recorded in the file **Set_bak.json** but changing settings in the panel does not edit the **Settings.py** file.

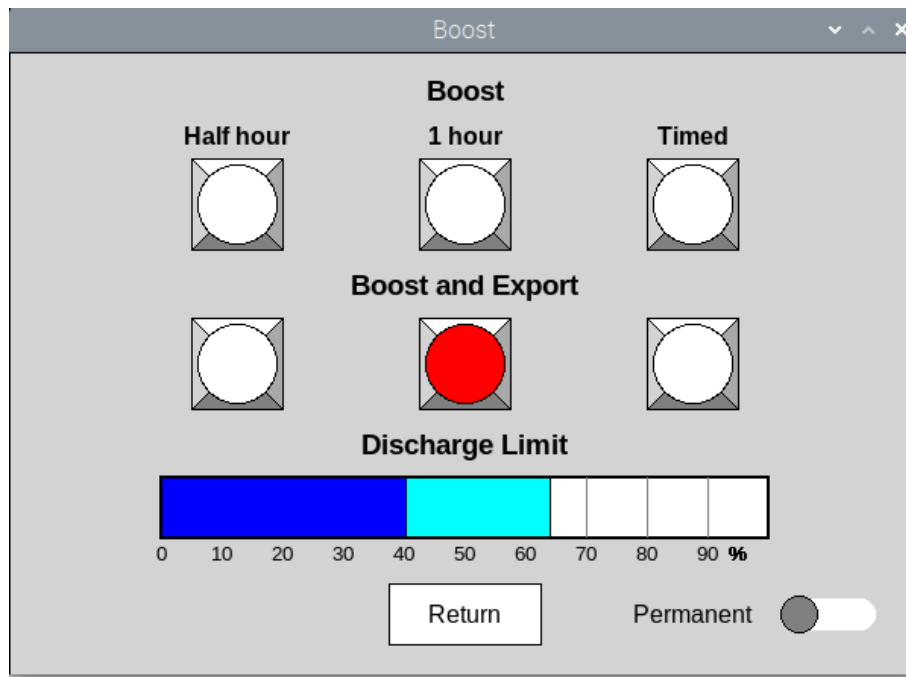
Using the Boost facility

Boost is a new function in Meter7.9.py

It is used to allow you to turn on battery power for a fixed period, when the battery is otherwise not discharging.

For example, you may have the battery discharge time set to 4 pm in order to use the batteries during the peak time in the evening. If you want to use something that is going to take electricity earlier in the day, say a tumble dryer, without taking power from the grid, you can use the Boost function to allow a time limited discharge.

To use Boost, click or press on the Timer icon  at the bottom left of the screen. This will open the Boost dialogue box.



If you press the first button, you will get an immediate* boost for 30 minutes.

If you press the second button you will get a boost lasting for one hour.

Pressing the third button will allow you to set a Boost to operate any time that you wish within the next 24 hours.

Set the time using the nudge panel that appears, then press **Return**.

A limit can be set on how much to allow the batteries to discharge. To set this, simply touch the relevant point on the bar at the bottom of the window. (Python unfortunately does not allow the point to be changed by sliding the finger/mouse).

The **Permanent** switch at the bottom right of the page can be turned on if you want the boost to repeat each day at the same time.

Note that you can only set the limit higher than the current battery state of charge if the **Permanent** switch is turned on.

Do not forget to press **Return** to close the Boost dialogue and allow the app to continue.

*Although I say immediate, sending data to the inverter may take up to 30 seconds, but it may take up to a minute before the change is reflected on the display.

If you want to end a Boost before it is finished, open the Boost dialogue and press the 'illuminated' button so that it returns to white, then press **Return**.

If you want to extend a Boost period, open the Boost dialogue and either press the illuminated button twice, or press an alternative button then press **Return**.

The second row is identical to the first, except that any power not used by the house will be exported to the grid.

As the boost functions are totally controlled by the script, the script must be running for the boost instructions to be sent to the inverter.

Note that the discharge limit only applies during the boost period and does not limit the discharge of the batteries after the boost session has finished.

Instructions for use of Octopus functions

Power Meter v.7.9 can now automatically detect an Octopus Saving Session and control the inverter during the session.

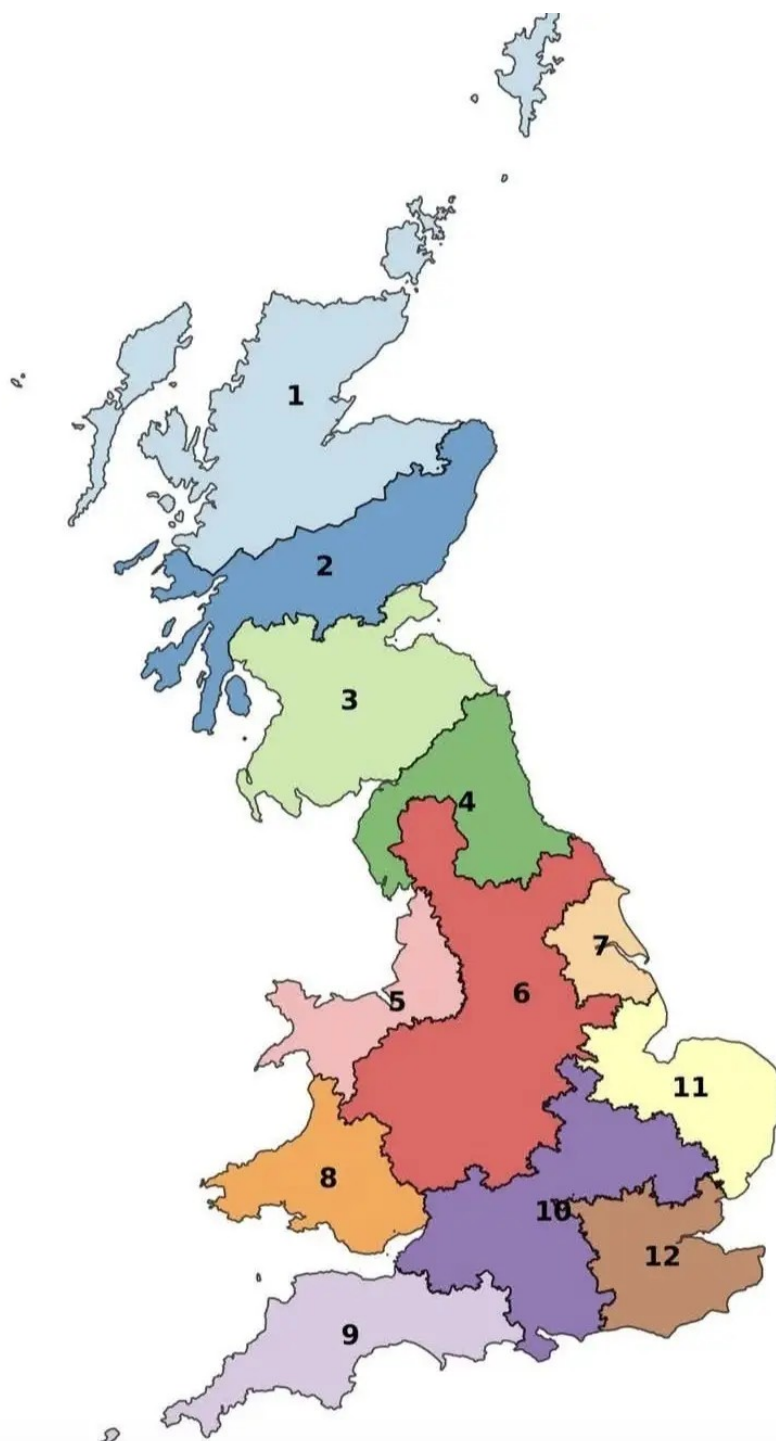
Power Meter detects when Octopus' bid to save power has been accepted by reading the National Grid (NESO) API. This does not guarantee that you will be offered a saving session, but one is at least taking place in your area..

As the bids are regionalised, Power Meter must know which region you live in. The regions have recently been reorganised and now have numbers rather than names.

If you wish to know which region you are in, view the map on the next page:

The appropriate region must be entered into **Settings.py**. Enter it as a number without quotation marks.

<code>octopus = "YES"</code>	<code>#set to "YES" to automatically reset battery discharge time if an Octopus Saving Session is happening</code>
<code>region = 10</code>	<code>#set to your National Grid region - see Octopus instructions</code>
<code>ssExport = "YES"</code>	<code>#set to "YES" to export during saving sessions</code>



When the saving session is detected, the discharge start time is adjusted to the start of the saving session. If **ssExport** is set to "YES", then the batteries will export during the saving session to increase the possible return. When the saving session is complete, the system will be returned to its normal automatic mode. Note that if you are normally setting the discharge start time manually, you will have to do so again after the saving session is complete. As well as the settings in **Settings.py**, the Octopus functions can be turned on and off in the Control Panel.

Octopus Free Sessions

At present, these cannot be automatically detected, but if you are offered one, you can turn the **Free Session** function on in the Control Panel. These sessions will normally be offered the day before, so when you receive one, turn the Control Panel option on and you will then be presented with a scroll panel to set the start and end times. Press return and then close the control panel.

The overnight preload, if set automatically, will be reduced to allow some space in the batteries ready for the free session. When the free session starts, the batteries will be switched to charge from the grid until either the session ends or the batteries reach 95% full. At this point the session will end and the status will return to normal.

The latest version of the NESO API, also allows for free sessions. So far none have occurred, but if one is offered, it will be set up exactly the same as the Octopus Free Sessions mentioned above.

Remember that the Octopus functions will only operate if you are connected to both your inverter and to the wider internet.

As the Octopus functions are totally controlled by the script, the script must be running during the session, otherwise the instructions will not be received by the inverter.

EV Mode

EV mode came about because a lot of people complain on social media that charging their car causes their house batteries to discharge.

There are ways around this by moving CT clamps on your solar power system, but that is not always possible, so I decided that there may be a way to solve it within the script.

EV mode relies on the fact that most things that take a high load do not do so for very long; ovens and heaters have thermostats, kettles are only on for a short time, so that most high loads will switch off very quickly if only for a moment. Car charging however is a large load which is sustained, and therefore different. That can be detected and Meter 7.9 attempts to distinguish between the the car charger and other household loads.

When car charging is detected, the battery discharge will be turned off until the charging has obviously ceased.

In settings you can set a power threshold and a delay before the car charging is detected. The correct setting will differ with your charging set up, but with a 7 kW charger, a good setting might be:

```
EVmode = "YES"
```

```
carThreshold = 7000
```

```
carDelay = 10
```

where the script turns off the battery discharge if there is a sustained load of over 7 kW for longer than ten minutes.

The delay should mean that you only lose slightly more than 1 kWh from your house battery rather than flattening it completely.

Alternatively, if you are using a plug-in 'granny' charger you might set:

```
EVmode = "YES"
```

```
carThreshold = 2000
```

```
carDelay = 20
```

where a sustained load of over 2 kW for more than 20 minutes will switch off the battery discharge. A larger load, such as a 5 kW oven, should not trigger the EV mode due to the thermostat periodically switching off the oven.

In practice this will be down to experimentation and what works best in your situation.

Bear in mind that some other items, particularly tumble dryers, may also cause the EV function to trigger. In this event you can effectively cancel EV mode by using the Boost function.

Either way, the purpose is to retain the maximum charge in your house battery until the evening peak.

EV mode only applies in daytime when power is at full price, EV mode is suppressed during the night when electricity is cheap. EV mode will also not operate if another function such as a Saving Session or Boost is operational.

Instructions for using Meter7.9.py with the new SolaX Pocket Dongle v4.0

With the new version of the pocket WiFi dongle, Solax have made some changes which have defeated the use of Power Meter via your house WiFi.

It is, however, possible to use the full functions of Meter 7.9 on a Raspberry Pi, but it requires a bit more work.

First, you will need a cable connection from your internet router to the Ethernet socket of your Raspberry Pi.

If your router is WiFi only, then this system will not work and you will be restricted to only the basic functions. It may be easier to use Meter6.9.py.

Having cable connected your Raspberry Pi to your router, you need to open the router's management page and make sure that the IP number for the SolaX dongle, 192.168.10.10, is outside the range that the router can 'see'. This will prevent any accidental IP clashes. The common IP for household routers: 192.168.1.1 and the subnet mask of 255.255.255.0 are ideal for this. If your router has an IP address in the range 192.168.10.1 to 192.168.10.254, then you will have to change it.

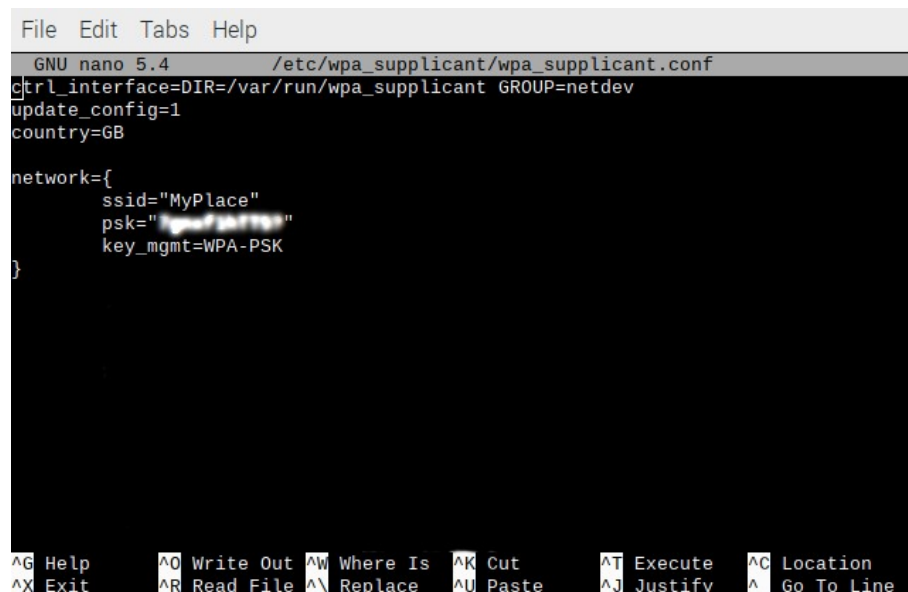
In the settings file for Meter 7.9, set the inverter address to 192.168.10.10

[illegible]

Next, on your Raspberry Pi, open the terminal and type

```
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

This should open a page looking like this:

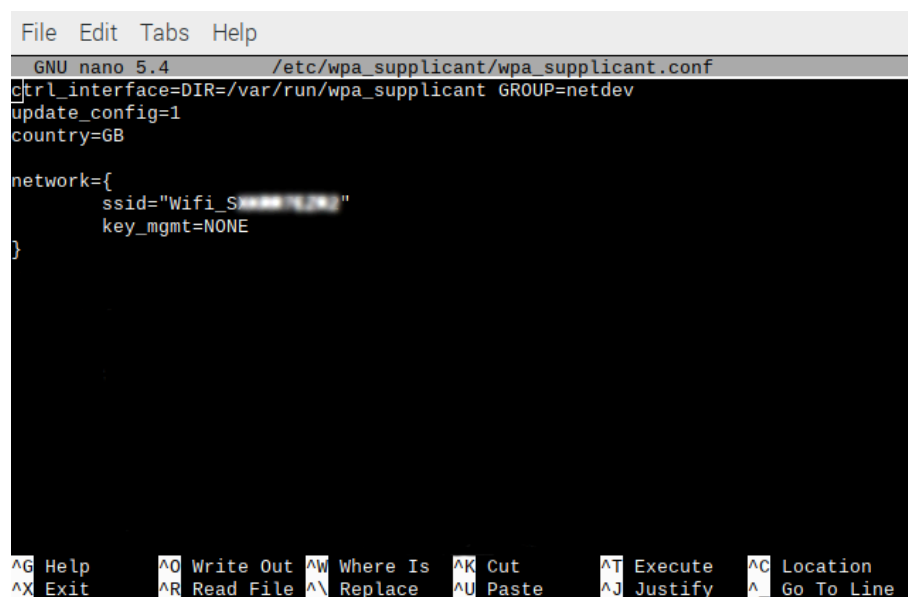


```
File Edit Tabs Help
GNU nano 5.4 /etc/wpa_supplicant/wpa_supplicant.conf
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
country=GB

network={
    ssid="MyPlace"
    psk="12345678901234567890"
    key_mgmt=WPA-PSK
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace  ^U Paste     ^J Justify  ^_ Go To Line
```

Edit the page to the following. Note that you will need to use the arrow keys on your keyboard to move around, as the mouse does not work on this page.



```
File Edit Tabs Help
GNU nano 5.4 /etc/wpa_supplicant/wpa_supplicant.conf
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
country=GB

network={
    ssid="Wifi_S[REDACTED]"
    key_mgmt=NONE
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace  ^U Paste     ^J Justify  ^_ Go To Line
```

where the letters after **Wifi_** are the registration number of your WiFi dongle Do not delete or modify the first three lines.

Go to your inverter and press the control button once on your inverter's WiFi dongle. This will start the dongle's access point (AP). This will only remain active for a few minutes before turning off again. While the AP is running, reboot your Raspberry Pi, and the Pi should connect to the dongle's AP.

This will be indicated by the symbol at the top of the Pi's screen, ringed in red in the image below.



Once the connection is made, the AP should remain active. Note that only one device at a time can access the AP, so local control on your phone app will not work while the Pi is connected.

Now run `Meter7.9.py`. I recommend running it from Thony the first time as that will enable you to see any problems. If all is well, the script should show the current status of the inverter.

I recommend, while Meter 7.9 is running, to open the built-in browser and make sure that you can still access the internet. If you can, then the Solcast and Octopus functions of Meter 7.9 should work.

If you wish to set up Meter 7.9 to run as an embedded system, you can now do so, but remember to press the control button on the SolaX dongle to restart the AP before rebooting the Pi.

All the above applies to the Raspberry Pi, but a similar method will work with other systems, including a PC, but you will have to use the appropriate commands for that system to access the inverter and internet simultaneously.

Instructions for using Meter6.9.py

Meter6.9.py has its settings in a separate file **Settings.py**.

This file can be edited with any text editor, but please take care and make sure that text strings are enclosed in quotation marks and that your text editor does not try to correct any spellings.

For details on editing this file, read **Instructions for Settings.py** earlier in this document.

Meter6.9.py displays PV power, battery power, excess power to the grid and the current battery state of charge.

It also displays the energy used from the grid during peak (daytime) and off-peak (nighttime) periods plus any exports back to the grid.

The pylon symbol shows whether power is currently going to or from the grid. A black pylon means that no power is flowing. The pylon will turn blue if power is being received from the grid and red if it is being exported.

With the Solcast module activated, the script will display an estimated overnight preload of the battery to make the best use of both PV power and battery power.

The script can create a log file called **powerlog.csv**, which can be entered into a spreadsheet program for further processing.

The values logged are Solar yield, Battery use, Grid power used during daytime, Grid power used during nighttime off-peak, and Exported energy to the grid over the previous 24 hours.

These values are not logged by the SolaX cloud, but are calculated by the script, therefore for accurate values, the script must run continuously.

Each 24 hours starts at the beginning of the off-peak period or midnight if no off-peak period is set. Note that when you first run the script, the first day's log may not be recorded or may be incomplete.

For users of Octopus Energy the script can also detect Octopus Savings Sessions, although unlike Meter 7.9 it cannot set the inverter, that will have to be done manually via either the SolaX Cloud website or the SolaX app.

The script creates two temporary files on your drive. **power.json** stores the details from the last access to the inverter and allows the script to resume quickly after a restart.

daymeans.json stores average daily power usage for the Solcast module.

The format of these files is now identical to those for Meter 7.9. so you can move seamlessly between the two scripts.

If updating from a version of the script prior to Meter 6.7, you must delete the old **power.json** file before running **Meter6.9.py** as the format has changed.

Meter6.9.py requires four external modules, **Solcast4.py**, **Week2.py**, **Octopus5.py** and **powerlog.py**, which must be in the same directory as the main script along with the graphics files. It also requires a special version of graphics.py by John Zelle, called **graphics3.py** which is included in the repository.

Error Messages

The Meter6.9.py and Meter7.9.py scripts have the ability to display error messages when a problem occurs. Here are the relevant messages and what they mean:

```
Unable to contact cloud
```

Meter6.9.py has lost contact with the SolaX Cloud

```
Unable to contact inverter
```

Meter7.9.py has lost contact with the inverter

Both these errors may only be temporary. The script will wait for 5 minutes then try again. If the message becomes permanent, check your WiFi and in the case of Meter6.9, your internet connection.

```
Too many failures contacting cloud/inverter
```

The cloud or the inverter is not returning valid data. The script will try 5 times to get valid data, and if it fails, this message will appear. Check that your inverter password and, in the case of Meter6.9, your API token, have been entered correctly.

```
Unable to contact Solcast
```

The script cannot access the Solcast website. Try the website in a browser to see if it is live:

<https://toolkit.solcast.com.au>

Remember that Meter 7.9 cannot access the internet when only connected directly to the SolaX inverter's access point.

```
Data invalid from Solcast
```

Solcast is not returning valid data. Check the Solcast Key and Resource ID from the Solcast website. Set up Solcast on the website if you have not already done so.

The following errors only apply to Meter7.7 onwards:

Inverter not found

Meter7.9 in DHCP mode has not been able to find the inverter on your local WiFi. This error may only be temporary. The script will wait for 5 minutes, then try again.
If the message becomes permanent, check that you are connected to the right WiFi network. Check that the router's DHCP range has been entered correctly.

Note: I have noticed that the inverter goes offline occasionally, perhaps when it is accessing the Cloud. In this case, this message may appear, but it will reconnect at the next attempt.

Wrong type of inverter

The script is not connected to a single-phase SolaX X1-Hybrid-G4 inverter. In this case, use Meter6.9.py instead.

Unable to set preload to...

For some reason, Meter7.9 has been unable to automatically set the preload value. Use the SolaX Cloud or the SolaX app to set the preload manually.

Discharge time not set

For some reason, Meter7.9 has been unable to set the battery discharge start time. Use the SolaX cloud or the SolaX app to check the current setting and change it if necessary.

Charge from grid disabled

Meter7.9 cannot set the preload as **Charge from grid** is disabled at the inverter.
Use the SolaX Cloud or the SolaX app to enable **Charge from grid** in **Self use** mode on the inverter. Make sure that **Work mode** is set to **Self use**.

Inverter fault

The inverter is returning that it is in a fault condition. Visit the SolaX Cloud website and go to the Alarms page (Bell icon at left). This is the warning log which will give you more details of the fault.

The following messages only applies to Meter6.9.py and Meter7.9.py

Corrupt backup file

The backup file is corrupt. As the back up file **Power.json** is written so often, this can occasionally happen due to no fault of the script. In this event the script will not display power from grid or exported power until valid results are available and a new back up file is created. That day's log will also not be written. If this error persists, it is time to replace your SD card as the card is becoming unreliable.

Using default settings

This error will occur if the settings back up file **Set_bak.json** is corrupted or is from an earlier version of the script. In this event the script will use the settings from the **Settings.py** file. Open the Control Panel to check your settings. This will create a new file.

